arXiv:2506.05587v1 [cs.AI] 5 Jun 2025

MMTU: A Massive Multi-Task Table Understanding and Reasoning Benchmark

Junjie Xing[*] University of Michigan
Yeye He[†] Microsoft Corporation
Mengyu Zhou Microsoft Corporation
Haoyu Dong Microsoft Corporation
Shi Han Microsoft Corporation
Lingjiao Chen Microsoft Corporation
Dongmei Zhang Microsoft Corporation
Surajit Chaudhuri Microsoft Corporation
H. V. Jagadish University of Michigan

Abstract

Tables and table-based use cases play a crucial role in many important real-world applications, such as spreadsheets, databases, and computational notebooks, which traditionally require expert-level users like data engineers, data analysts, and database administrators to operate. Although LLMs have shown remarkable progress in working with tables (e.g., in spreadsheet and database copilot scenarios), comprehensive benchmarking of such capabilities remains limited. In contrast to an extensive and growing list of NLP benchmarks, evaluations of table-related tasks are scarce, and narrowly focus on tasks like NL-to-SQL and Table-QA, overlooking the broader spectrum of real-world tasks that professional users face. This gap limits our understanding and model progress in this important area.

In this work, we introduce MMTU, a large-scale benchmark with over 30K questions across 25 real-world table tasks, designed to comprehensively evaluate models ability to understand, reason, and manipulate real tables at the expert-level. These tasks are drawn from decades' worth of computer science research on tabular data, with a focus on complex table tasks faced by professional users. We show that MMTU require a combination of skills – including table understanding, reasoning, and coding – that remain challenging for today's frontier models, where even frontier reasoning models like OpenAI o4-mini and DeepSeek R1 score only around 60%, suggesting significant room for improvement. We highlight key findings in our evaluation using MMTU and hope that this benchmark drives further advances in understanding and developing foundation models for structured data processing and analysis. Our code and data are available at `https://github.com/MMTU-Benchmark/MMTU` and `https://huggingface.co/datasets/MMTU-benchmark/MMTU`.

1 Introduction

Remarkable progress has been made in foundation models [39, 40, 68, 112, 33], partly thanks to an expanding array of large-scale benchmarking efforts. Prominent examples include benchmarks for general language understanding (e.g., GLUE [114], Super-GLUE [115], BIG-Bench [108], MMLU [73], MMLU-pro [118]), as well as benchmarks focused on coding and STEM reasoning

[1]Correspondence: jjxing@umich.edu
[2]Correspondence: yeyehe@microsoft.com

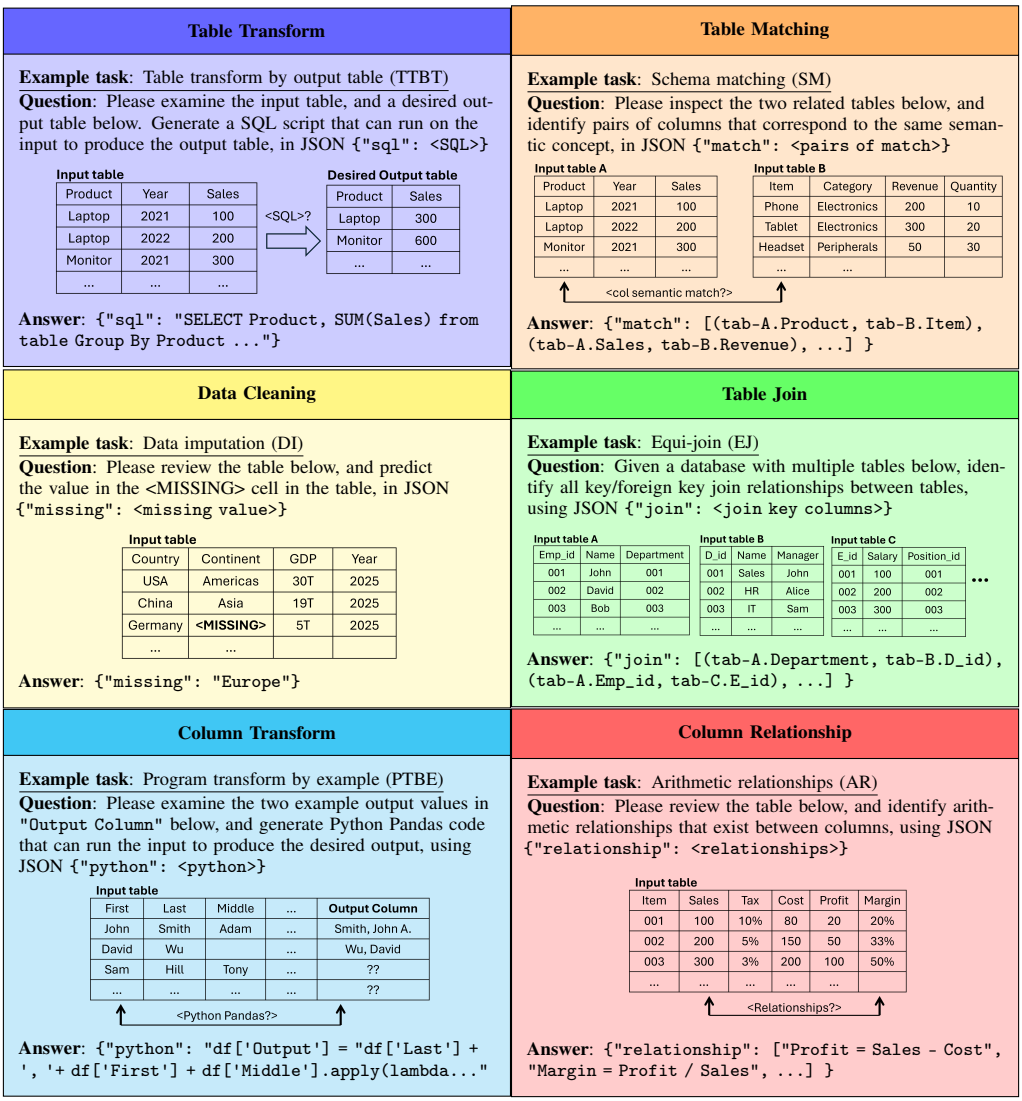

Figure 1: Example questions sampled from different task categories in MMTU for illustrative purposes. Note that questions in MMTU all follow a standardized triplet format of: `(Instruction, Input-table(s), Ground-truth answer)`. The list of all tasks can be seen in Table 1.

(e.g., SWE-bench [13], GPQA-diamond [103], AIME [5], LiveCodeBench [11]). These efforts have played a critical role in deepening our understanding and accelerating the progress of foundation models.

Tables and table-based use cases are central to many real-world applications, including spreadsheets [21, 2], databases [17, 18], and computational notebooks [25, 19], which often require expert-level users such as data engineers, analysts, and database administrators to operate. While LLMs have shown great promise in working with tables [22, 16, 24], existing evaluations of table-tasks remain narrow in scope – primarily focusing on NL-to-SQL [86, 137, 128] and Table-QA [127, 54, 49] – and fail to reflect the broader spectrum of real-world tasks that professional users face.

In this work, we introduce MMTU, a large-scale and challenging benchmark comprising 30,647 questions across 25 real-world table tasks. It is designed to comprehensively evaluate models' ability to understand, reason, and manipulate real tables at the expert-level. The tasks we collect in MMTU are drawn from decades of computer science research on tabular data, with a particular focus on complex tasks that professional users face, as illustrated in Figure 1.

Our evaluation shows that the complex and technical nature of MMTU demands a combination of capabilities – including table understanding, reasoning, and coding – that remain challenging for today's frontier models. Even the top-performing models, such as OpenAI o4-mini and DeepSeek

Table 1: Tasks and datasets in the MMTU benchmark. Most of these tasks have not traditionally been used to evaluate foundation models (except NL-to-code, Table QA, and KB mapping).

Task Category	Task Name	Task Description	Metric	References and Datasets	# Questions
Table Transform	table-transform-by-relationalization (TTBR)	Relationalize a table using transformation	Acc	[87, 35, 78]	238
	table-transform-by-output-schema (TTBS)	Synthesize transformation by output schema	Acc	[125]	700
	table-transform-by-output-table (TTBT)	Synthesize transformation by input/output tables	Acc	[36, 116, 113, 132]	87
Table Matching	Entity matching (EM)	Match rows refer to the same semantic entity	Acc	[61, 97, 136, 89, 99]	4380
	Schema matching (SM)	Match columns refer to the same concept	F1	[81, 135, 37, 94]	723
	Head value matching (HVM)	Match column-headers with cell-values	Acc	[88, 102]	952
Data Cleaning	data-imputation (DI)	Predict missing values in tables	Acc	[41, 64, 88]	2000
	error-detection (ED)	Detect erroneous cells in tables	F1	[56, 117, 95, 46]	1987
	list-to-table (L2T)	Split lists of undelimited values into table	Acc	[55, 42, 65, 84]	1000
Table Join	semantic-join (SJ)	Predict semantic join between two tables	Acc	[71, 31, 60, 119]	131
	equi-join-detect (EJ)	Predict equi-joins between a set of tables	F1	[90, 52, 105, 76]	517
Column Transform	program-transform-by-example (PTBE)	Program transformation by input/output examples	Acc	[72, 66, 63, 107]	568
	formula-by-context (FBC)	Predict formula based on table context	Acc	[48, 50]	3626
	semantic-transform-by-example (STBE)	Predict semantic transformations by examples	Acc	[71, 31, 60]	131
Column Relationship	arithmetic-relationship (AR)	Predict arithmetic-relationship (AR) in tables	F1	[106, 1]	819
	functional-relationship (FR)	Predict functional-relationship (FR) in tables	F1	[1, 100, 120]	309
	string-relationship (SR)	Predict string-relationship (SR) in tables	F1	[1, 72, 67]	766
Table understanding	Needle-in-a-haystack-table (NIHT)	Retrieve cell content in a table	Acc	[64]	1000
	Needle-in-a-haystack-index (NIHI)	Retrieve index based on cell value	Acc	[64]	1000
NL-2-code	NL-2-SQL (NS)	Translate natural-language into SQL	Acc	[137, 86, 83, 128, 138]	3289
Table QA	Table Question Answering (TQA)	Answer questions based on tables	Acc	[122, 127, 53, 54]	2424
	Fact Verification (FV)	Verify facts based on tables	Acc	[49, 126, 131]	1000
KB Mapping	Column type annotation (CTA)	Predict KB types based on column content	Acc	[77, 124, 109, 130, 75]	1000
	Column property annotation (CPA)	Predict KB property for a pair of columns	Acc	[77, 59, 98]	1000
	Cell entity annotation (CEA)	Predict KB entity for a table cell	Acc	[77, 62, 38, 121, 92]	1000
Total					30,647

, achieve only 63.9% and 59.6% on MMTU, suggesting substantial room for improvement, and highlighting MMTU as a strong testbed for models aspiring toward general human-level intelligence, e.g., to meet or surpass the top 10% of skilled adults in diverse technical tasks like characterized in [96].

We perform extensive experiments benchmarking a large collection of models using MMTU, and performed extensive analysis. Some of the key findings from our evaluation include:

- LLMs demonstrate strong potential in understanding and manipulating tabular data. Newer and larger models substantially outperform older and smaller ones, indicating significant advancements in table-related capabilities as captured by MMTU.
- Reasoning-focused models, such as DeepSeek and OpenAI o4-mini, show a clear advantage over general-purpose chat models like GPT-4o and Llama. The top reasoning models outperform the top chat models by over 10 percentage points (Table 3), underscoring the complexity of the tasks (which often require coding in SQL/Pandas) and the importance of reasoning in MMTU.
- Unlike earlier models, today's frontier models are less sensitive to how tables are formatted and serialized (e.g., markdown/CSV/JSON/HTML), reflecting general progress in models' abilities in understanding diverse data formats (Figure 8).
- LLMs still struggle with long table context, or large tables with many rows and columns. Complex tasks requiring holistic reasoning across cell values, especially in the column direction, remains challenging when the table context is long (Figure 6 and Figure 12).
- LLM performance can degrade under table-level perturbations such as row or column shuffling, even when these changes are supposed to be semantically invariant in the context of tables (Figure 7). This sensitivity points to possible limitations in models' ability to understand tables in a robust manner.

We hope MMTU can serve as a valuable addition to the growing landscape of model benchmarks, helping to track progress, identify limitations, and ultimately drive further advancements in this important area of using LLMs for table tasks.

2 Related Work

Large-scale benchmarks for foundation models. The rapid advancement of foundation models has made benchmark evaluation ever more important. Prominent large-scale benchmarks, such as GLUE [114] (9 NLP tasks), Super-GLUE [115] (10 NLP tasks), Big-BENCH [108] (204 tasks), MMMU [73] (15,908 questions), MMMU-pro [118] (12,032 questions), MMLU [129] (11,550 multi-modal questions) etc., offer comprehensive evaluations of model capabilities. However, as models improve rapidly, benchmarks can become saturated quickly (e.g., in the matter of a few years), prompting newer and more challenging benchmarks [115, 118]. All of these benchmarking efforts have nevertheless played a crucial role in measuring and stimulating the development of foundation models.

To contribute to this growing landscape, our new MMTU benchmark comprises 30,647 challenging questions across diverse table tasks that expert users would face, which is comparable in scale with prior efforts such as MMMU and MMLU. It complements existing benchmarks, by enabling comprehensive evaluation of foundation models in the important yet underexplored area of table reasoning and understanding.

Benchmarks for reasoning. Reasoning has recently emerged as a key challenge for foundation models. Beyond general intelligence benchmarks (e.g., GPQA Diamond [103], AGIEval [139], HLE [8]), there are specialized benchmarks targeting mathematical reasoning (e.g., AIME [5], MathVista [12], IMO [9]), coding (e.g., SWE-bench [13], CodeForce [7], LiveCodeBench [11], IOI [10]), and multi-modal reasoning (e.g., MMMU [129], ARC [6]). These benchmarks have become important tools for evaluating models' ability to tackle complex reasoning tasks, but can also get saturated quickly (e.g., GSM8k [57], Math500 [74], HuamEval [45]), making it necessary to create new and more challenging benchmarks.

We show that our MMTU benchmark can serve to complement existing reasoning benchmarks, by enabling evaluation on complex table-based tasks that require two-dimensional table understanding, coding, and logical reasoning. MMTU extends current reasoning benchmarks into the important yet underexplored domain of tabular data, which underpins many real-world applications.

Existing benchmarks relating to tables. Given the importance of table data, benchmarks have been developed in the ML and NLP community to evaluate model ability on tables, which however usually focus on a small set of table tasks such as NL-2-SQL [137, 86, 83, 128, 138] and Table-QA [49, 122, 127, 53, 54]. In contrast, MMTU expands the scope of current evaluations by incorporating 19 additional table tasks drawn from decades of research in communities such as data management and programming languages, resulting in a more comprehensive evaluation framework for assessing LLM capabilities on tabular data.

More recently, spreadsheet-centric benchmarks have emerged, including Spreadsheet-Bench [93], SheetCopilotBench [85], and Sheet-RM [51]. While these are important in the spreadsheet domain, these efforts are typically limited in scale (containing a few hundred cases), and are closely tied to specific file formats (e.g., .xlsx) and software environments. In comparison, MMTU focuses on general-purpose tabular data that applies broadly across spreadsheet, database, and computational notebook settings, enabling more scalable and format-independent evaluation of foundation models.

3 MMTU Benchmark for Tables

3.1 Benchmark overview

We introduce our Massive Multi-task Table Understanding and Reasoning (MMTU) benchmark, designed to evaluate models table understanding and reasoning capabilities at the expert-level, across a wide range of real-world tasks that would typically be performed by professional data engineers, data scientists, and database administrators.

The benchmark comprises 30,647 complex table-centric questions over 67,886 real tables, in 25 distinct task categories. Each question has a standardized format of "`<Instruction, Input-Table(s), Ground-truth answer>`", like illustrated in example questions in Figure 1. Detailed statistics of the benchmark can be found in Table 2, which highlight the diversity and complexity of the questions in MMTU.

These questions are meticulously collected and curated based on decades of computer science research in areas beyond ML/NLP – such as data management and programming languages – drawing on expert-labeled datasets developed over many years by researchers in these communities, as we will detail below which we will describe below.

3.2 Data curation workflow

Figure 2 illustrates the key steps in the overall workflow of our data curation process for producing MMTU. We detail each step in turn below.

Literature survey. To ensure our table tasks reflect real-world challenges, we draw on our experience working on related problems, that many challenging predictive table tasks have been studied in the

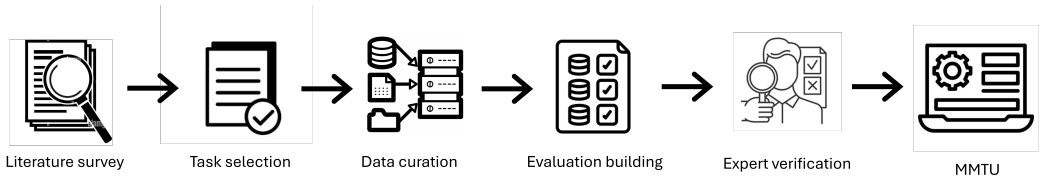

Figure 2: MMTU data curation workflow: we survey real-world table tasks from the literature, select 25 user-facing tasks with objective evaluation criteria, curate questions from 52 datasets, develop evaluation scripts and verify the results for these tasks, before arriving at the MMTU benchmark.

Statistics	**Number**
Total Questions	30,647
Total tasks / datasets	25/52
Total tables	67,886
Coding Questions	8,508 (27.8%)
- SQL questions	3,289 (10.7%)
- Python Pandas questions	1,593 (5.2%)
- Spreadsheet Formula questions	3,626 (11.8%)
Non-coding Questions	22,139 (72.2%)
Questions with tables	30,647
- Questions with 1 table	21,696 (70.8%)
- Questions with 2 tables	5,565 (18.2%)
- Questions with 3 or more tables	3,386 (11.0%)
Table characteristics	
- Average table row count	2,659
- Average table column count	11
- Average table cell count	33,251
Table sources	
- Web tables	48,502 (71.4%)
- Spreadsheet tables	4,716 (6.9%)
- Relational tables	14,668 (21.6%)

Table 2: Statistics of benchmark questions

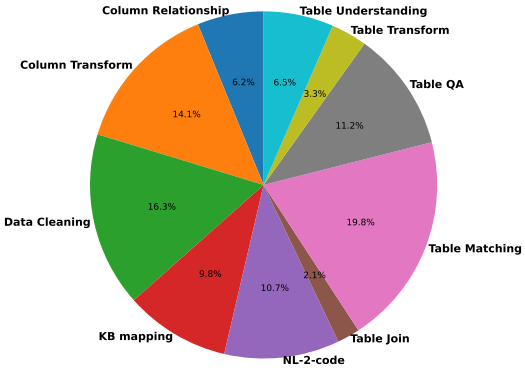

Figure 3: Question distribution by task category

decades worth of computer science research, particularly in data management (SIGMOD/VLDB), programming languages (PLDI/POPL), and web data (WWW/WSDM) communities. We conduct a systematic survey of publications from these venues over the past two decades, leveraging a combination of keyword searches of paper titles, and DeepResearch-like tools [20] (where we specify detailed requirements for papers in these venues), to arrive at a promising set of papers and possible candidate tasks.

Task selection. We manually examine candidate tasks described in the surveyed papers from the previous step, and select tasks that are:

(1) Real user-facing tasks, involving data tables that would otherwise require expert-level humans to perform. (We therefore exclude system-level predictive table tasks focused on performance improvements, such as query optimization [80, 82] and cardinality estimation [69, 79]);
(2) Objectively evaluable tasks, that come with unique manually-labeled ground truth. (We therefore exclude tasks such as table summarization [134, 34, 70, 101, 44] and table augmentation [133, 123], which lack unique ground-truth and may require subjective fuzzy LLM-based evaluations);
(3) Tasks based on real-world data tables, which can be real web tables, spreadsheet tables, or relational tables, etc. (We exclude tasks and datasets based on synthetic or perturbed data).
After the selection step, we arrive at 25 different tasks (listed in Table 1) from 52 diverse benchmark datasets (can be seen in Table 4 in the appendix). We note that a majority of these real-world tasks (all except the last 3 categories in Table 1, NL-2-code, Table-QA and KB-mapping) have not been used to evaluate foundation models. A summary of these tasks are described in more detail in Appendix A.

Data standardization and curation. To accommodate the heterogeneity across the 52 benchmark datasets (which have diverse data formats, ground-truth labels, and task definitions), we next standardize questions in each dataset into a consistent "`<Instruction, Table(s), Ground-truth>`" format. Figure 1 shows examples of the triplet format for different tasks. This enables consistent representation across tasks, facilitating the integration of diverse table tasks within a single benchmark framework, and allowing different LLMs to be plugged in for easy model prediction and evaluation.

Out of an abundance of caution, in this step we also prompt GPT-4o to filter out any instance of benchmark questions that may raise privacy or security concerns. Additionally, to preserve the diversity of the overall benchmark, we cap the number of questions contributed by any single dataset at 1000. The final composition of questions is reported in Table 1, with more statistics shown in Table 2 and Figure 3.

Evaluation framework. In contrast to benchmarks like MMLU [73] and MMMU [129], which primarily use multiple-choice formats (where evaluation involves comparing a predicted option such as "A/B/C/D" against a single-letter ground truth), real-world table tasks performed by professional experts are often far more complex and nuanced. These tasks, such as code generation or structured reasoning, cannot be adequately evaluated using multiple-choice alone. In MMTU, we instead adopt a structured yet open-ended answer format (see Figure 1 for examples) for prediction and evaluation.

To support evaluations beyond simple string comparisons, we designed a lightweight evaluation framework that supports diverse evaluations in table tasks, including execution-based evaluation (for SQL and Python generation), and structured output evaluation (e.g., comparing an unordered JSON list against ground truth). Our framework is also extensible, making it easy to incorporate new task types and evaluation metrics. Details of our evaluation framework can be seen in [15].

Expert verification. As a final verification step, we sample 20 questions per task and employ domain experts with years of experience to manually review and verify that (1) raw data is integrated correctly, (2) the ground-truth aligns with human intuition and passes verification, (3) the reference instruction properly reflects the task to produce the desired output, and (4) the evaluation script is set up correctly to correctly evaluate model predictions against ground-truth.

After completing all data curation steps in the workflow illustrated in Figure 2, we obtain a total of 30,647 questions that form our MMTU benchmark, with main statistics summarized in Table 2.

3.3 Broader discussions

Limitations. A main limitations of our benchmark is how our tasks are sampled and selected. Like discussed in Section 3.2, for ease of evaluation, we include only tasks that can be objectively evaluated, and omit ones that are subjective or creative in nature (e.g., table summarization, generation, and enrichment) that are also important to users, but not included in the benchmark.

In addition, since we sampled table tasks from the existing research literature, which naturally introduces biases, as it omits tasks that are important in practice but not well studied in the literature, or tasks that lack good labeled data (e.g., multi-turn table manipulation).

Lastly, while human experts often read tables visually on two dimensional grid (which makes two dimensional spatial reasoning easy), our current evaluations only use text-based input, and do not consider multi-modal input. Extending the benchmark to multi-modal table input is an interesting direction for future work.

Broader impacts. Our benchmark is designed to evaluate LLMs performance on challenging expert-level table tasks, with the goal of identifying model shortcomings and stimulating model improvements. We hope this can lead to more capable models, to better assist human users in scenarios such as spreadsheet-copilot and database-copilot.

We make our best efforts to exclude any content that may raise privacy or security concerns, contain explicit material, depict violence, or be otherwise sensitive. For example, we manually review instructions and datasets at the task level, and employ GPT-4o at the data record level, to remove any that may have privacy concerns, in order to minimize potential negative effect of the data.

4 Experiments

Experimental setup. In all our experiments reported below, we use publicly available model endpoints for inference [4, 23] with default parameter settings. All of our code and data are publicly available at [15] for future research.

Model Type	Model	MMTU result	Cost per question (US$)
Chat	GPT-4o (2024-11-20)	0.491 ± 0.01	0.0204
	GPT-4o-mini (2024-07-18)	0.386 ± 0.01	0.0024
	Llama-3.3-70B	0.438 ± 0.01	0.0027
	Llama-3.1-8B	0.259 ± 0.01	0.0008
	Mistral-Large-2411	0.430 ± 0.01	0.0108
	Mistral-Small-2503	0.402 ± 0.01	0.0006
Reasoning	o4-mini (2024-07-18)	**0.639 ± 0.01**	0.0099
	Deepseek-R1	0.596 ± 0.01	0.0052

Table 3: Overall performance and cost results of frontier models.

4.1 Overall performance

We benchmark a range of frontier open-source and proprietary models using MMTU. Table 3 gives an overview of their performance, as well as their cost information. Notably, the two reasoning-focused models, OpenAI o4-mini and DeepSeek R1, perform the best at 63.9% and 59.6% respectively, significantly outperforming the chat-based models and underscoring the challenging nature of MMTU. Our analysis of the reasoning traces generated by R1 reveals that reasoning models excel on MMTU because of their strong coding skills and their ability to break complex tasks over large tables, into a sequence of more manageable sub-tasks on smaller table context (i.e., subsets of rows and columns relevant to the task at hand).

From a cost-efficiency perspective[1], both reasoning models are also competitive (despite producing more intermediate thinking tokens), due to the relatively light-weight nature of R1 and o4-mini, making them cost-effective choices for tackling table tasks.

In the rest of this paper, we will focus on four best-performing frontier models: two reasoning models (DeepSeek R1 and OpenAI o4-mini), and two general-purpose models (GPT-4o and Llama 3.3 70B), for detailed analysis to avoid clutter.

Figure 5 provides a detailed comparison across 10 task categories. While reasoning models outperform chat-based models, the relative empty nature of the radar chart reveals substantial gaps in model performance, highlighting opportunities for improvement. For instance, we can see that models still face difficulties with table-centric coding tasks (e.g., Column Transform, Table Transform, etc.). Tasks that require holistic reasoning across multiple tables and multiple columns (e.g., Table Join, Column Relationship, Data Cleaning, etc.) also remain challenging. We will present an error-analysis in Section 4.3.

A more granular breakdown across all 25 individual tasks is shown in Figure 4, where reasoning models (represented by shaded bars) noticeably outperform chat models on many complex tasks. Additional dataset-level performance results can be found in Table 4 in the appendix.

4.2 Detailed analysis and sensitivity

We highlight key analysis in this section, including long contexts, robustness to table perturbations, and sensitivity to format variations.

Long table context. Figure 6 shows the impact of long table context on model performance. In this analysis, we bucketize all questions within each task into four quartiles based on the token length of the associated tables. We then evaluate model accuracy within each quartile and aggregate the results across tasks, as shown in the figure. Across all four frontier models, performance consistently declines as the table context length increases (from left to right within each group).

These findings suggest that, despite recent advances in long-context LLMs [47, 32], long contexts remain a significant challenge for tables. In Appendix C, we use a detailed experimental comparison between the standard "needle-in-a-haystack (NIH)" [3, 104], and our table-based "needle-in-a-haystack in table (NIHT)" test – while frontier models perform nearly perfectly on NIH, their performance drops sharply on NIHT, revealing fundamental limitations in their ability to handle long table contexts.

Robustness to table permutation. Figure 7 illustrates model performance under different table permutations. Specifically, we randomly shuffle rows and/or columns in input tables[2], with the 4 bars

[1]Price-per-token figures are sourced from the respective API providers [27**? ? ? ? ?** , 28, 30].

[2]In shuffling columns, we keep the 3 left-most columns in all tables intact, as these are likely the "key columns" or "entity columns" in tables [43, 41] in tables, to best preserve the meaning of these tables.

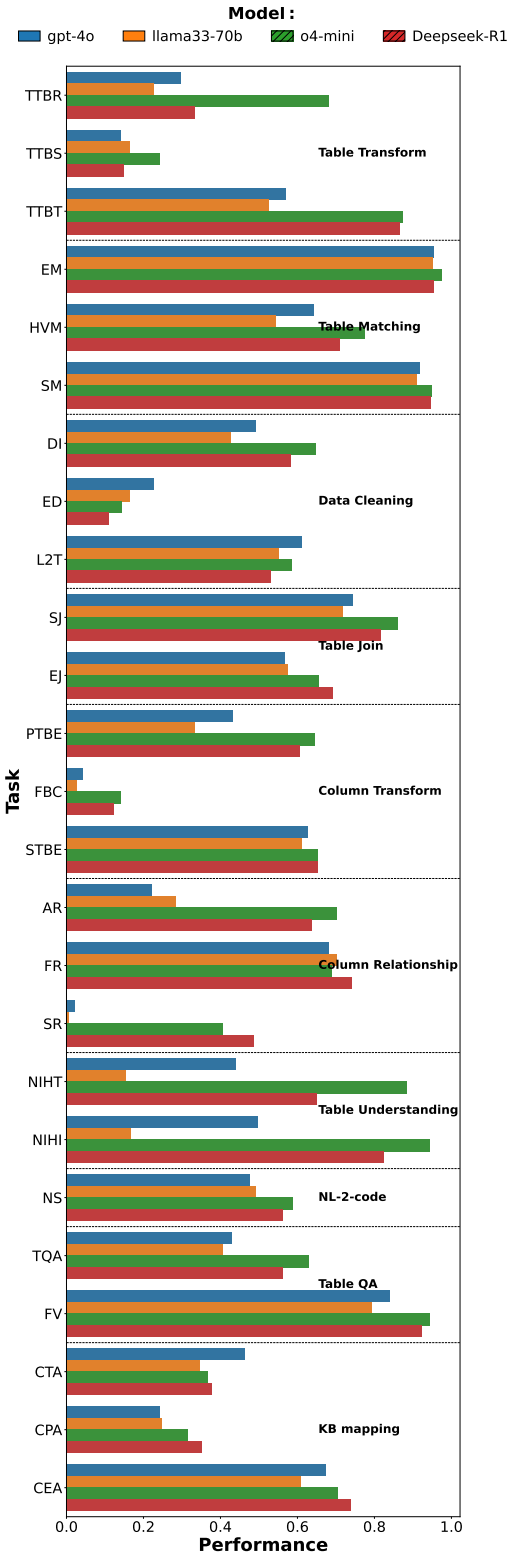

Figure 4: Performance comparison in all 25 tasks, for chat-models GPT-4o and Llama-3 (solid bars), as well as reasoning models DeepSeek and OpenAI o4-mini (shaded bars). Reasoning models are noticeably better on average.

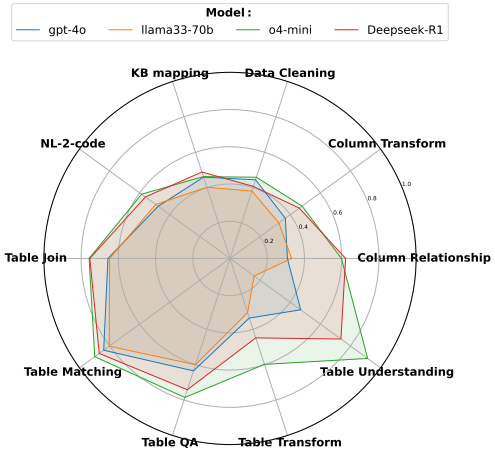

Figure 5: Performance comparison of frontier models in all 10 task-categories.

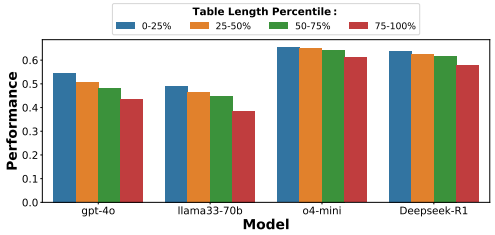

Figure 6: Impact of long table context on model performance.

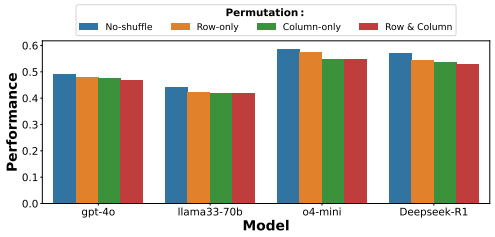

Figure 7: Impact of table-level permutation on model performance.

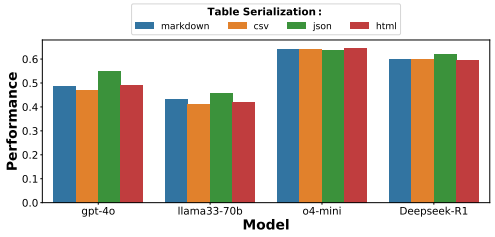

Figure 8: Impact of table format (markdown/csv/json/html) on model performance.

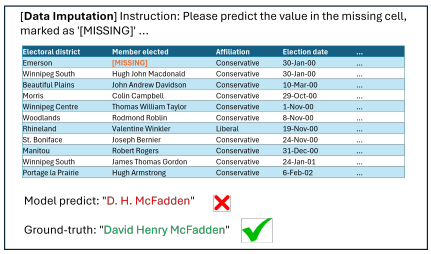

Figure 9: An example "table understanding" error in Data Imputation: while the model knows the person's name for the missing cell, it misses the table context and uses an abbreviated form of the name, instead of the full-name in the ground-truth.

in each group representing: (1) no shuffle, (2) row-only shuffle, (3) column-only shuffle, and (4) both row and column shuffle. Recall that unlike natural-language text, two dimension relational tables are permutation-invariant [88, 58], meaning that shuffling rows and columns should generally not change their semantic meanings and the associated tasks (e.g., the tasks in Figure 1 will remain the same, even if the rows/columns in the associated tables are shuffled).

However, the results show a consistent decline in model performance as we move from no permutation to row, column, and full (row + column) shuffling. Notably, column shuffling leads to a steeper decline than row shuffling. This suggests that despite their capabilities, language models that are pretrained primarily on linear, left-to-right text can remain sensitive to the structural ordering of tables, indicating a lack of robustness to alternate but semantically equivalent table layouts.

Table format variations. Figure 8 shows different models' performance using common table input formats: Markdown, CSV, JSON, and HTML. We observe that unlike prior studies that showed notable sensitivity of LLMs [110] to table formats, our results indicate that today's frontier models, especially reasoning ones (o4-mini and R1), are becoming less sensitive to these format variations, reducing the need for format-specific optimization as models continue to improve. For chat models (GPT-4o and Llama-3.3), we note that using JSON still has an advantage on MMTU, mainly because it is easier for models to identify value/column correspondence in the JSON format, especially in long table context settings (e.g., in needle-in-a-haystack tasks like NIHT and NIHI shown in Table 1).

4.3 Error analysis

We sampled 10 questions from each task, to manually analyze the underlying reason of these errors. We categorize all model errors into 4 main categories below:

Table understanding (38%). Table understanding is the largest category of errors in our analysis. Figure 9 shows a simple example from the Data Imputation task that is intuitive to understand. While the model correctly identifies the person's name for the missing cell, it filled in the value "`D. H. McFadden`" that is in an abbreviated form, inconsistent with other values in the same column that use the full-name format (the correct answer should be "`David Henry McFadden`").

We observe that models are prone to errors when handling long table contexts, such as multiple tables or large tables with many rows and columns, as illustrated in Figure 6. Figure 10 shows a concrete example of the issue in the task of table-reshaping (TTBR in Table 1), where the reasoning trace shows that the model miscalculates the column index of the target column. This issue of long-context table is also notable in tasks like List-to-Table (L2T), Data Imputation (DI), Equi-Join (EJ), and NL-to-SQL (NS), etc.

A more detailed analysis of challenges in long-context tables is provided in Appendix C, where we examine a table-specific variant of the "needle-in-a-haystack" task, to understand why long-context tables remain challenging for models.

Reasoning and coding (28%). We find models can often make mistakes on tasks that require coding and reasoning on top of tables. For instance, for tasks in Table Transform, Column Transform, and NL-2-Code categories, models can generate code (SQL or Pandas) that is "close" to the correct answer, but misses important details that require reasoning over table context holistically (e.g., data format across all cells in the same column), which leads to incorrect results.

Knowledge (18%). For tasks in the categories of KB mapping (CEA and CTA), Semantic Join (SJ), and Data Imputation (DI), we find models can sometimes hallucinate facts (e.g., in the case of CEA,

[**Data Reshaping**] Instruction: Please predict table reshaping transformations for the table below ...

Series appearances	Team	Wins	Losses	Win %	Season(s)
40	New York Yankees	27	13	0.675	1921, 1922, 1923, 1926, 1927, 1928
20	San Francisco Giants	8	12	0.4	1905, 1911, 1912, 1913, 1917, 1921
19	St. Louis Cardinals	11	8	0.579	1926, 1928, 1930, 1931, 1934, 1942
18	Los Angeles Dodgers (	6	12	0.333	1916, 1920, 1941, 1947, 1949, 1952
14	Oakland Athletics (Ka	9	5	0.643	1905, 1910, 1911, 1913, 1914, 1929
12	Boston Red Sox (Amer	8	4	0.667	1903, 1912, 1915, 1916, 1918, 1946
11	Detroit Tigers	4	7	0.364	1907, 1908, 1909, 1934, 1935, 1940
5	New York Mets	2	3	0.4	1969, 1973, 1986, 2000, 2015
4	Kansas City Royals	2	2	0.5	1980, 1985, 2014, 2015
2	Toronto Blue Jays	2	0	1	1992, 1993
2	Miami Marlins	2	0	1	1997, 2003

Model reason: ... The explode transformation requires the explode_column_idx parameter. The "Season(s)" column is the fifth column, (index 4 if we count from 0). So [{'operator': 'explode', 'explode_column_idx': 4}] ✗

Ground-truth: [{'operator': 'explode', 'explode_column_idx': 5}] ✓

Figure 10: An example "table understanding" error in Table Reshaping: while the model knows the correct transforms to perform ("explode" in Pandas), it miscounts the column index for the target column "Season(s)", which should be the sixth column instead of the fifth based on the reasoning trace, which leads to an incorrect transformation.

creating knowledge base references that do not exist), or recite inaccurate facts (e.g., in the case of Semantic Join and Data Imputation).

Other (15%). The remaining issues, such as result extraction, timeout and context limitation in response generation, and possible ambiguity in questions/ground-truth, fall in this category.

5 Conclusion

In this work, we present MMTU, a comprehensive benchmark designed to assess foundation models on a broad range of real-world table tasks. By focusing on real-world tasks that professional data engineers, data analysts, and database administrators often have to face, MMTU poses expert-level challenges for foundation models in table understanding and reasoning.

Future work includes broadening the scope of table tasks to cover those that are important in practice but underrepresented in the existing research literature, as well as incorporating more subjective or creative tasks such as data generation, summarization, and enrichment. Another promising direction is the integration of multi-modal evaluation.

References

[1] Auto-relate: A unified approach to discovering reliable functional relationships leveraging statistical tests. https://drive.google.com/drive/folders/1opO-tglvyJ6_yG37ZbMob57dLRjBAuMI.

[2] Google sheets. https://workspace.google.com/products/sheets/.

[3] Llm test: Needle in a haystack. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

[4] Azure OpenAI inference. https://learn.microsoft.com/en-us/azure/ai-services/openai/reference.

[5] Aime: American invitational mathematics examination. https://maa.org/math-competitions/american-invitational-mathematics-examination-aime, .

[6] Arc challenge. https://arcprize.org/, .

[7] codeforce competition. https://codeforces.com/contests, .

[8] Humanity's last exam. https://agi.safe.ai/, .

[9] Imo: International mathematics olympiad questions for model testing. https://openai.com/index/introducing-openai-o1-preview/, .

[10] Ioi: International olympiad in informatics questions for model testing. https://openai.com/index/learning-to-reason-with-llms/, .

[11] Livecodebench. https://livecodebench.github.io/, .

[12] Mathvista: Evaluating math reasoning in visual contexts. https://mathvista.github.io/, .

[13] Swe-bench. https://www.swebench.com/, .

[14] Claude 3 release annoucement. https://www.anthropic.com/news/claude-3-family?utm_source=chatgpt.com.

[15] MMTU: Code and data. Benchmark data: https://huggingface.co/MMTU-benchmark, Evaluation code: https://github.com/MMTU-Benchmark/MMTU.

[16] Database copilot. https://learn.microsoft.com/en-us/azure/azure-sql/copilot/copilot-azure-sql-overview?view=azuresql, .

[17] Oracle. https://www.oracle.com/, .

[18] Sql server. https://www.microsoft.com/en-us/sql-server, .

[19] Databricks notebooks. https://www.databricks.com/product/collaborative-notebooks, .

[20] Arc challenge. https://openai.com/index/introducing-deep-research/.

[21] Microsoft excel. https://www.microsoft.com/en-us/microsoft-365/excel, .

[22] Excel copilot. https://support.microsoft.com/en-us/office/get-started-with-copilot-in-excel-d7110502-0334-4b4f-a175-a73abdfc118a, .

[23] Models endpoints in Azure AI Foundry. https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/models-featured#cohere.

[24] Gemini in bigquery. https://cloud.google.com/gemini/docs/bigquery/overview.

[25] Jupyter notebooks. https://jupyter.org/.

[26] The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[27] DeepSeek r1: price info (Retrieved in 05/2025). https://api-docs.deepseek.com/quick_start/pricing, .

[28] GPT-4o: price info (Retrieved in 05/2025). https://platform.openai.com/docs/pricing, .

[29] Llama 3: price info (Retrieved in 05/2025). https://azuremarketplace.microsoft.com/en-us/marketplace/apps/metagenai.llama-3-3-70b-instruct-offer?tab=PlansAndPrice, .

[30] o4-mini: price info (Retrieved in 05/2025). https://platform.openai.com/docs/pricing, .

[31] Ziawasch Abedjan, John Morcos, Ihab F Ilyas, Mourad Ouzzani, Paolo Papotti, and Michael Stonebraker. Dataxformer: A robust transformation discovery system. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pages 1134–1145. IEEE, 2016.

[32] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[33] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

[34] Junwei Bao, Duyu Tang, Nan Duan, Zhao Yan, Yuanhua Lv, Ming Zhou, and Tiejun Zhao. Table-to-text: Describing table region with natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[35] Daniel W Barowy, Sumit Gulwani, Ted Hart, and Benjamin Zorn. Flashrelate: extracting relational data from semi-structured spreadsheets using examples. *ACM SIGPLAN Notices*, 50 (6):218–228, 2015.

[36] Rohan Bavishi, Caroline Lemieux, Roy Fox, Koushik Sen, and Ion Stoica. Autopandas: neural-backed generators for program synthesis. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–27, 2019.

[37] Zohra Bellahsene, Angela Bonifati, Fabien Duchateau, and Yannis Velegrakis. *On evaluating schema matching and mapping*. Springer, 2011.

[38] Chandra Sekhar Bhagavatula, Thanapon Noraset, and Doug Downey. Tabel: Entity linking in web tables. In *International Semantic Web Conference*, pages 425–441. Springer, 2015.

[39] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[40] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[41] Michael J Cafarella, Alon Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. Webtables: exploring the power of tables on the web. *Proceedings of the VLDB Endowment*, 1(1):538–549, 2008.

[42] Michael J Cafarella, Alon Y Halevy, Yang Zhang, Daisy Zhe Wang, and Eugene Wu. Uncovering the relational web. In *WebDB*, pages 1–6. Citeseer, 2008.

[43] Kaushik Chakrabarti, Surajit Chaudhuri, Zhimin Chen, Kris Ganjam, Yeye He, and W Redmond. Data services leveraging bing's data assets. *IEEE Data Eng. Bull.*, 39(3):15–28, 2016.

[44] Jieying Chen, Jia-Yu Pan, Christos Faloutsos, and Spiros Papadimitriou. Tsum: fast, principled table summarization. In *Proceedings of the Seventh International Workshop on Data Mining for Online Advertising*, pages 1–9, 2013.

[45] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[46] Qixu Chen, Yeye He, Raymond Chi-Wing Wong, Weiwei Cui, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. Auto-test: Learning semantic-domain constraints for unsupervised error detection in tables. *arXiv preprint arXiv:2504.10762*, 2025.

[47] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

[48] Sibei Chen, Yeye He, Weiwei Cui, Ju Fan, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. Auto-formula: Recommend formulas in spreadsheets using contrastive learning for table representations. *Proceedings of the ACM on Management of Data*, 2(3): 1–27, 2024.

[49] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. Tabfact: A large-scale dataset for table-based fact verification. *arXiv preprint arXiv:1909.02164*, 2019.

[50] Xinyun Chen, Petros Maniatis, Rishabh Singh, Charles Sutton, Hanjun Dai, Max Lin, and Denny Zhou. Spreadsheetcoder: Formula prediction from semi-structured context. In *International Conference on Machine Learning*, pages 1661–1672. PMLR, 2021.

[51] Yibin Chen, Yifu Yuan, Zeyu Zhang, Yan Zheng, Jinyi Liu, Fei Ni, and Jianye Hao. Sheetagent: A generalist agent for spreadsheet reasoning and manipulation via large language models. In *ICML 2024 Workshop on LLMs and Cognition*, 2024.

[52] Zhimin Chen, Vivek Narasayya, and Surajit Chaudhuri. Fast foreign-key detection in microsoft sql server powerpivot for excel. *Proceedings of the VLDB Endowment*, 7(13):1417–1428, 2014.

[53] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, et al. Finqa: A dataset of numerical reasoning over financial data. *arXiv preprint arXiv:2109.00122*, 2021.

[54] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. Hitab: A hierarchical table dataset for question answering and natural language generation. *arXiv preprint arXiv:2108.06712*, 2021.

[55] Xu Chu, Yeye He, Kaushik Chakrabarti, and Kris Ganjam. Tegra: Table extraction by global record alignment. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pages 1713–1728, 2015.

[56] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*, pages 2201–2206, 2016.

[57] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[58] Tianji Cong, Madelon Hulsebos, Zhenjie Sun, Paul Groth, and HV Jagadish. Observatory: Characterizing embeddings of relational tables. *arXiv preprint arXiv:2310.07736*, 2023.

[59] Vincenzo Cutrona, Jiaoyan Chen, Vasilis Efthymiou, Oktie Hassanzadeh, Ernesto Jiménez-Ruiz, Juan Sequeda, Kavitha Srinivas, Nora Abdelmageed, Madelon Hulsebos, Daniela Oliveira, et al. Results of semtab 2021. *Proceedings of the semantic web challenge on tabular data to knowledge graph matching*, 3103:1–12, 2022.

[60] Arash Dargahi Nobari and Davood Rafiei. Dtt: An example-driven tabular transformer for joinability by leveraging large language models. *Proceedings of the ACM on Management of Data*, 2(1):1–24, 2024.

[61] Sanjib Das, AnHai Doan, Paul Suganthan G. C., Chaitanya Gokhale, Pradap Konda, Yash Govind, and Derek Paulsen. The magellan data repository. `https://sites.google.com/site/anhaidgroup/projects/data`.

[62] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. Turl: Table understanding through representation learning. *ACM SIGMOD Record*, 51(1):33–40, 2022.

[63] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. Robustfill: Neural program learning under noisy i/o. In *International conference on machine learning*, pages 990–998. PMLR, 2017.

[64] Gus Eggert, Kevin Huo, Mike Biven, and Justin Waugh. Tablib: A dataset of 627m tables with context. *arXiv preprint arXiv:2310.07875*, 2023.

[65] Hazem Elmeleegy, Jayant Madhavan, and Alon Halevy. Harvesting relational tables from lists on the web. *Proceedings of the VLDB Endowment*, 2(1):1078–1089, 2009.

[66] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

[67] Sumit Gulwani, William R Harris, and Rishabh Singh. Spreadsheet data manipulation using examples. *Communications of the ACM*, 55(8):97–105, 2012.

[68] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[69] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, et al. Cardinality estimation in dbms: A comprehensive benchmark evaluation. *arXiv preprint arXiv:2109.05877*, 2021.

[70] Braden Hancock, Hongrae Lee, and Cong Yu. Generating titles for web tables. In *The World Wide Web Conference*, pages 638–647, 2019.

[71] Yeye He, Kris Ganjam, and Xu Chu. Sema-join: joining semantically-related tables using big table corpora. *Proceedings of the VLDB Endowment*, 8(12):1358–1369, 2015.

[72] Yeye He, Xu Chu, Kris Ganjam, Yudian Zheng, Vivek Narasayya, and Surajit Chaudhuri. Transform-data-by-example (tde) an extensible search engine for data transformations. *Proceedings of the VLDB Endowment*, 11(10):1165–1177, 2018.

[73] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[74] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[75] Madelon Hulsebos, Kevin Hu, Michiel Bakker, Emanuel Zgraggen, Arvind Satyanarayan, Tim Kraska, Çagatay Demiralp, and César Hidalgo. Sherlock: A deep learning approach to semantic data type detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1500–1508, 2019.

[76] Lan Jiang and Felix Naumann. Holistic primary key and foreign key detection. *Journal of Intelligent Information Systems*, 54(3):439–461, 2020.

[77] Ernesto Jiménez-Ruiz, Oktie Hassanzadeh, Vasilis Efthymiou, Jiaoyan Chen, and Kavitha Srinivas. Semtab 2019: Resources to benchmark tabular data to knowledge graph matching systems. In *The Semantic Web: 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31–June 4, 2020, Proceedings 17*, pages 514–530. Springer, 2020.

[78] Zhongjun Jin, Michael R Anderson, Michael Cafarella, and HV Jagadish. Foofah: Transforming data by example. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 683–698, 2017.

[79] Kyoungmin Kim, Jisung Jung, In Seo, Wook-Shin Han, Kangwoo Choi, and Jaehyok Chong. Learned cardinality estimation: An in-depth study. In *Proceedings of the 2022 international conference on management of data*, pages 1214–1227, 2022.

[80] Jan Kossmann, Thorsten Papenbrock, and Felix Naumann. Data dependencies for query optimization: a survey. *The VLDB Journal*, 31(1):1–22, 2022.

[81] Christos Koutras, George Siachamis, Andra Ionescu, Kyriakos Psarakis, Jerry Brons, Marios Fragkoulis, Christoph Lofi, Angela Bonifati, and Asterios Katsifodimos. Valentine: Evaluating matching techniques for dataset discovery. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 468–479. IEEE, 2021.

[82] Hai Lan, Zhifeng Bao, and Yuwei Peng. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. *Data Science and Engineering*, 6: 86–101, 2021.

[83] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. Kaggledbqa: Realistic evaluation of text-to-sql parsers. *arXiv preprint arXiv:2106.11455*, 2021.

[84] Kristina Lerman, Craig Knoblock, and Steven Minton. Automatic data extraction from lists and tables in web sources. In *IJCAI-2001 Workshop on Adaptive Text Extraction and Mining*, volume 98, 2001.

[85] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAO-XIANG ZHANG. Sheetcopilot: Bringing software productivity to the next level through large language models. *Advances in Neural Information Processing Systems*, 36:4952–4984, 2023.

[86] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36, 2024.

[87] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. Auto-tables: Synthesizing multi-step transformations to relationalize tables without using examples. *arXiv preprint arXiv:2307.14565*, 2023.

[88] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. Table-gpt: Table-tuned gpt for diverse table tasks. *arXiv preprint arXiv:2310.09263*, 2023.

[89] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. *arXiv preprint arXiv:2004.00584*, 2020.

[90] Yiming Lin, Yeye He, and Surajit Chaudhuri. Auto-bi: Automatically build bi-models leveraging local join prediction and global schema graph. *arXiv preprint arXiv:2306.12515*, 2023.

[91] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[92] Jixiong Liu, Yoan Chabot, Raphaël Troncy, Viet-Phi Huynh, Thomas Labbé, and Pierre Monnin. From tabular data to knowledge graphs: A survey of semantic table interpretation tasks and methods. *Journal of Web Semantics*, 76:100761, 2023.

[93] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: towards challenging real world spreadsheet manipulation. *arXiv preprint arXiv:2406.14991*, 2024.

[94] Jayant Madhavan, Philip A Bernstein, and Erhard Rahm. Generic schema matching with cupid. In *vldb*, volume 1, pages 49–58, 2001.

[95] Mohammad Mahdavi and Ziawasch Abedjan. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment*, 13(12): 1948–1961, 2020.

[96] Meredith Ringel Morris, Jascha Sohl-Dickstein, Noah Fiedel, Tris Warkentin, Allan Dafoe, Aleksandra Faust, Clement Farabet, and Shane Legg. Levels of agi for operationalizing progress on the path to agi. *arXiv preprint arXiv:2311.02462*, 2023.

[97] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 international conference on management of data*, pages 19–34, 2018.

[98] Phuc Nguyen, Ikuya Yamada, Natthawut Kertkeidkachorn, Ryutaro Ichise, and Hideaki Takeda. Semtab 2021: Tabular data annotation with mtab tool. In *SemTab@ ISWC*, pages 92–101, 2021.

[99] George Papadakis, Ekaterini Ioannou, Emanouil Thanos, and Themis Palpanas. *The four generations of entity resolution*. Springer, 2021.

[100] Marcel Parciak, Sebastiaan Weytjens, Niel Hens, Frank Neven, Liesbet M Peeters, and Stijn Vansummeren. Measuring approximate functional dependencies: A comparative study. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3505–3518. IEEE, 2024.

[101] Ankur P Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. *arXiv preprint arXiv:2004.14373*, 2020.

[102] Erhard Rahm and Philip A Bernstein. A survey of approaches to automatic schema matching. *the VLDB Journal*, 10:334–350, 2001.

[103] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

[104] Jonathan Roberts, Kai Han, and Samuel Albanie. Needle threading: Can llms follow threads through near-million-scale haystacks? *arXiv preprint arXiv:2411.05000*, 2024.

[105] Alexandra Rostin, Oliver Albrecht, Jana Bauckmann, Felix Naumann, and Ulf Leser. A machine learning approach to foreign key discovery. In *WebDB*, 2009.

[106] Roee Shraga and Renée J Miller. Explaining dataset changes for semantic data versioning with explain-da-v. *Proceedings of the VLDB Endowment*, 16(6), 2023.

[107] Rishabh Singh. Blinkfill: Semi-supervised programming by example for syntactic string transformations. *Proceedings of the VLDB Endowment*, 9(10):816–827, 2016.

[108] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.

[109] Yoshihiko Suhara, Jinfeng Li, Yuliang Li, Dan Zhang, Çağatay Demiralp, Chen Chen, and Wang-Chiew Tan. Annotating columns with pre-trained language models. In *Proceedings of the 2022 International Conference on Management of Data*, pages 1493–1503, 2022.

[110] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 645–654, 2024.

[111] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[112] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[113] Quoc Trung Tran, Chee-Yong Chan, and Srinivasan Parthasarathy. Query by output. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*, pages 535–548, 2009.

[114] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.

[115] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. *Advances in neural information processing systems*, 32, 2019.

[116] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. Synthesizing highly expressive sql queries from input-output examples. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 452–466, 2017.

[117] Pei Wang and Yeye He. Uni-detect: A unified approach to automated error detection in tables. In *Proceedings of the 2019 International Conference on Management of Data*, pages 811–828, 2019.

[118] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

[119] Yue Wang and Yeye He. Synthesizing mapping relationships using table corpus. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1117–1132, 2017.

[120] Ziheng Wei and Sebastian Link. Discovery and ranking of functional dependencies. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1526–1537. IEEE, 2019.

[121] Tianxing Wu, Shengjia Yan, Zhixin Piao, Liang Xu, Ruiming Wang, and Guilin Qi. Entity linking in web tables with multiple linked knowledge bases. In *Semantic Technology: 6th Joint International Conference, JIST 2016, Singapore, Singapore, November 2-4, 2016, Revised Selected Papers 6*, pages 239–253. Springer, 2016.

[122] Xianjie Wu, Jian Yang, Linzheng Chai, Ge Zhang, Jiaheng Liu, Xeron Du, Di Liang, Daixin Shu, Xianfu Cheng, Tianzhen Sun, et al. Tablebench: A comprehensive and complex benchmark for table question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25497–25506, 2025.

[123] Mohamed Yakout, Kris Ganjam, Kaushik Chakrabarti, and Surajit Chaudhuri. Infogather: entity augmentation and attribute discovery by holistic matching with web tables. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 97–108, 2012.

[124] Cong Yan and Yeye He. Synthesizing type-detection logic for rich semantic data types using open-source code. In *Proceedings of the 2018 International Conference on Management of Data*, pages 35–50, 2018.

[125] Junwen Yang, Yeye He, and Surajit Chaudhuri. Auto-pipeline: synthesizing complex data pipelines by-target using reinforcement learning and search. *arXiv preprint arXiv:2106.13861*, 2021.

[126] Xiaoyu Yang and Xiaodan Zhu. Exploring decomposition for table-based fact verification. *arXiv preprint arXiv:2109.11020*, 2021.

[127] Yi Yang, Wen-tau Yih, and Christopher Meek. Wikiqa: A challenge dataset for open-domain question answering. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 2013–2018, 2015.

[128] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.

[129] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9556–9567, 2024.

[130] Dan Zhang, Yoshihiko Suhara, Jinfeng Li, Madelon Hulsebos, Çağatay Demiralp, and Wang-Chiew Tan. Sato: Contextual semantic type detection in tables. *arXiv preprint arXiv:1911.06311*, 2019.

[131] Hongzhi Zhang, Yingyao Wang, Sirui Wang, Xuezhi Cao, Fuzheng Zhang, and Zhongyuan Wang. Table fact verification with structure-aware transformer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1624–1629, 2020.

[132] Sai Zhang and Yuyin Sun. Automatically synthesizing sql queries from input-output examples. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 224–234. IEEE, 2013.

[133] Shuo Zhang and Krisztian Balog. Entitables: Smart assistance for entity-focused tables. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*, pages 255–264, 2017.

[134] Shuo Zhang, Zhuyun Dai, Krisztian Balog, and Jamie Callan. Summarizing and exploring tabular data in conversational search. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1537–1540, 2020.

[135] Yu Zhang, Mei Di, Haozheng Luo, Chenwei Xu, and Richard Tzong-Han Tsai. Smutf: Schema matching using generative tags and hybrid features. *arXiv preprint arXiv:2402.01685*, 2024.

[136] Chen Zhao and Yeye He. Auto-em: End-to-end fuzzy entity-matching using pre-trained deep models and transfer learning. In *The World Wide Web Conference*, pages 2413–2424, 2019.

[137] Danna Zheng, Mirella Lapata, and Jeff Z Pan. Archer: A human-labeled text-to-sql dataset with arithmetic, commonsense and hypothetical reasoning. *arXiv preprint arXiv:2402.12554*, 2024.

[138] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.

[139] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. Agieval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364*, 2023.

A Task overview

A.1 Column transformation

Column transformation refers to the category of tasks, where new columns in a table are derived using existing columns. We select three key tasks in this category:

Program Transform by Example (PTBE) [72, 66, 63, 107]. This is a popular task studied in the data management and programming language literature. In this task, we are given a few example output values in a new output column (usually provided by users as demonstrations), and the task is to synthesize a transformation program using input tables, such that the produced output can match the user-given output examples. Figure 1 shows an example of this task. We use the benchmark datasets from [72, 66] for this task in MMTU.

Semantic Transform by example (STBE) [71, 31, 60]. This task is similar to PFBE above, in that users also provide a few example output to demonstrate the intended transformations, but the target transformation requires "semantic" transformations (e.g., country to capital-city, or company to stock-ticker), that cannot be programmed using a syntactic program like in PFBE. We use the datasets from [71, 31]for our benchmark.

Formula by Context (FBC) [48, 50]. This task is studied in the context of spreadsheets, where given a target spreadsheet cell in which users want to enter a spreadsheet formula, we are asked to predict the intended formula in the target cell. We use the 4 benchmark datasets in [48].

A.2 Table Transformation

In the table transformation task category, we also need to perform transformations, but unlike "Column transformations" above where only new columns are derived (without changing the shape of the input table), in "Table transformation", a broader class of transformations can be invoked (e.g., table reshaping, restructuring, group-by, aggregation, etc.), which can change the form and shape of the original input table.

Table Transform by Output Table (TTBT) [36, 116, 113, 132]. In this task, we are given a pair of input/output tables, and the task is to infer a transformation program (in SQL or Pandas), which when executed on the input table, can generate the desired output table. Figure 1 shows an example of this task. We use the datasets from [36, 116].

Table Transform by Output Schema (TTBS) [125]. This task is similar to TTBT above, except that the output table is a schematic depiction of how the desired output table should look like, without being the exact output that the desired transformation program would generate on the given input (as it is sometimes hard to generate such output table without first producing the desired program). We use the dataset in [125]for this task.

Table Transform by Relationalization (TTBR) [87, 35, 78]. Because relational data analysis often requires input tables to be in a relational form, this task transforms data in non-relational semi-structured forms, into the standard relational form. The task is to predict such a relationalization transformation program, based on the characteristics of the input table. We use the dataset in [87].

A.3 Table matching

In this task category, we try to match relevant rows and columns between multiple tables.

Entity Matching (EM) [61, 97, 136, 89, 99]. Entity matching is the task of determining whether rows or entities from two tables correspond to the same real-world entity. We use the datasets in [61, 97]for benchmarking.

Schema Matching (SM) [81, 135, 37, 94]. Schema matching tries to match relevant columns from two tables that map to the same semantic concept. Figure 1 shows an example of this task. We use datasets in [81, 135]for MMTU.

Header Value Matching (HVM) [88, 102]. In HVM, we are given a table without headers, and a shuffled list of the original headers in the table, where the task is to match column headers with column values in tables. We use the dataset in [88]for benchmarking.

A.4 Data cleaning

Tasks in the data cleaning category tries to improve the quality of input tables, which are popular tasks studied in the data management literature.

Data Imputation (DI) [41, 64, 88]. Data imputation is the intuitive task of predicting missing values in a relational table, based on the surrounding context of the table. Figure 1 shows an example of this task. We use the dataset from [41, 64].

Error Detection (ED) [56, 117, 95, 46]. The task of Error detection aims to identify erroneous values in a table that are semantically inconsistent or anomalous with the rest of the column. We use the dataset in [46].

List to Tables (L2T) [55, 42, 65, 84]. In List-to-table, the task is to segment records of data without clear separators, into columns, so that values in the same column are homogeneous, and the resulting table becomes a natural relational table, with values consistently aligned in homogenous columns. We use the dataset from [55].

A.5 Table join

Join is an important operation that connects multiple related tables together.

Equal Join (EJ) [90, 52, 105, 76]. In Equal-join, we are given a collection of related tables, and the task is to identify all join relationships between the tables. Figure 1 shows an example of this task. We use the dataset in [90].

Semantic Join (SJ) [71, 31, 60, 119]. In Semantic Join, we are also tasked to join related tables together, but instead of the common equi-join, which is based on string-equality comparisons, the join relationship is based on semantic relatedness (e.g., country and capital city, or company and stock ticket). We use the same semantic-transformation datasets from [71, 31], but converting the input/output transformation columns, as join keys from two separate tables.

A.6 Column relationships

In this category of tasks, the goal is to identify implicit but semantically meaningful relationships from an input table.

Arithmetic Relationship (AR) [106, 1]. This task focuses on identifying arithmetic relationships from an underlying table. Figure 1 shows an example of this task.

String Relationship (SR) [1, 72, 67]. This task focuses on identifying string transformation relationships from a given table.

Functional Relationship (FR) [1, 100, 120]. This task focuses on identifying functional-relationships from input tables. We use the datasets from [1] our as benchmarks for all three tasks.

A.7 Table understanding

Tasks in this category are intended to test models ability to understand tables, and retrieve relevant facts from tables.

Needle-in-a-haystack-table (NIHT). In this task, we create a variant of the popular "Needle-in-a-haystack" task in the context of tables, like described in detail in Appendix C. We use the dataset from [64]to construct tests in this task.

Needle-in-a-haystack-index (NIHI). In this task, we reverse the NIHT task above, and ask models to identify index positions corresponding to a given value in a table. We use the same dataset from [64]to construct tests for this task.

A.8 NL-2-Code

NL-2-SQL (NS) [137, 86, 83, 128, 138]. NL-2-SQL is a popular task to translate natural-language questions into SQL statements that can execute given an input table. We use the benchmarks from [137, 86, 83, 128, 138]in MMTU.

A.9 Table Question Answering (Table QA)

Table-QA (TQA) [122, 127, 53, 54]. Table QA is another popular task, where models are used to directly answer questions on a given input table, without code execution. We use benchmarks from [49, 122, 127, 53]for this task.

Fact verification (FV) [49, 126, 131]. Table fact verification is a variant of TQA, in which models are asked if a statement is refuted or supported by facts presented in an input table. We use data from [49]for this task.

A.10 Knowledge-base mapping (KB Mapping)

KB mapping is the task where we map facts and relationships of a table, to known KB ontologies.

Column type annotation (CTA) [77, 124, 109, 130, 75]. This is a popular task where columns in a table is mapped to known entity types in a knowledge-base. We use the CTA dataset from [77].

Column property annotation (CPA) [77, 59, 98]. [77]. The CPA task is to predict the relationship of a given pair of columns, based on known properties in a knowledge-base. We use the CPA dataset from [77].

Cell entity annotation (CEA) [77, 62, 38, 121, 92]. The CEA task predicts the knowledge-base entity id of a given cell in a table. We use the CEA dataset from [77].

Table 4: Dataset-level Performance of Frontier Chat & Reasoning Models

Task Name	Dataset	Chat Model		Reasoning Model	
		GPT-4o	Llama33-70B	o4-mini	Deepseek-R1
Data-transform-reshape	Auto-Tables	0.298	0.227	**0.681**	0.332
Transform-by-output-target-schema	commercial-pipelines	0.077	0.154	**0.231**	0.077
	github-pipelines	0.204	0.176	**0.250**	0.220
	Average	0.140	0.165	**0.241**	0.148
Transform-by-input-output-table	AutoPandas	0.593	0.519	**0.815**	**0.815**
	Scythe	0.550	0.533	**0.933**	0.917
	Average	0.571	0.526	**0.874**	0.866
Entity-Matching	Amazon-Google	0.912	0.907	**0.918**	0.916
	BeerAdvo-RateBeer	0.943	0.932	**0.989**	0.955
	DBLP-ACM	0.992	0.975	0.988	**0.990**
	DBLP-Scholar	0.962	0.961	**0.968**	0.957
	Fodors-Zagats	0.989	0.989	**0.995**	0.989
	Walmart-Amazon	0.967	0.949	0.968	**0.974**
	iTunes-Amazon	0.915	0.953	**0.991**	0.887
	Average	0.954	0.952	**0.974**	0.953
header-value-matching	TableGPT	0.644	0.545	**0.774**	0.708
Schema-Matching	DeepMDatasets	**1.000**	**1.000**	**1.000**	**1.000**
	HXD	0.930	0.919	**0.950**	0.943
	Wikidata	**0.912**	0.881	**0.912**	**0.912**
	assays	0.825	0.818	0.902	**0.924**
	miller2	0.924	**0.947**	0.936	0.916
	prospect	0.919	0.893	**0.986**	0.972
	Average	0.918	0.910	**0.948**	0.944
Data-Imputation	WebTable	0.455	0.431	0.539	**0.546**
	tablib	0.532	0.422	**0.757**	0.616
	Average	0.494	0.426	**0.648**	0.581
Error-Detect	Relational-Tables	**0.266**	0.176	0.168	0.140
	Spreadsheet-Tables	**0.188**	0.152	0.120	0.078
	Average	**0.227**	0.164	0.144	0.109
List-to-table	TEGRA	**0.612**	0.552	0.584	0.531
semantic-join	DataXFormer	0.676	0.640	**0.819**	0.730
	SEMA-join	0.812	0.795	0.899	**0.901**
	Average	0.744	0.718	**0.859**	0.815
equi-join-detect	Auto-BI	0.567	0.576	0.655	**0.691**
Data-transform-pbe	TDE	0.398	0.318	**0.636**	0.581
	Transformation-text	0.467	0.349	**0.651**	0.630
	Average	0.433	0.334	**0.643**	0.605
Formula-prediction-context	cisco-random	0.065	0.032	**0.175**	0.127
	enron-random	0.034	0.017	**0.122**	0.101
	pge-random	0.030	0.050	0.113	**0.130**
	ti-random	0.041	0.013	**0.154**	0.129
	Average	0.042	0.028	**0.141**	0.122
semantic-transform	DataXFormer	0.393	0.360	0.427	**0.444**
	SEMA-join	0.861	0.865	**0.877**	0.863
	Average	0.627	0.613	0.652	**0.653**
Arithmetic-Relationship	Auto-Relate	0.221	0.284	**0.702**	0.638
Functional-Dependency	Auto-Relate	0.683	0.702	0.689	**0.741**
String-Relationship	Auto-Relate	0.021	0.007	0.405	**0.485**
Table-needle-in-a-haystack	experiment8	0.441	0.154	**0.884**	0.650
Table-Locate-by-Row-Col	experiment5	0.498	0.167	**0.944**	0.823
NL2SQL	Archer	0.212	0.154	**0.442**	0.375
	KaggleDBQA	0.292	0.492	**0.508**	0.503
	Spider	0.732	0.766	**0.774**	0.756
	WikiSQL	**0.745**	0.663	0.734	0.720
	bird	0.405	0.393	**0.478**	0.456
	Average	0.477	0.494	**0.587**	0.562
Table-QA	FinQA	0.196	0.147	**0.488**	0.361
	TableBench	0.410	0.420	**0.575**	0.554
	WikiQA	0.682	0.653	**0.819**	0.769
	Average	0.429	0.407	**0.627**	0.561
Table-Fact-Verification	TabFact	0.841	0.794	**0.943**	0.922
Column-type-annotation	SemTab2019	**0.463**	0.347	0.367	0.376
Columns-property-anotation	SemTab2019	0.242	0.249	0.315	**0.351**
Cell-entity-annotation	SemTab2019	0.673	0.610	0.705	**0.738**

B Detailed model performance at the dataset level

Table 4 shows a detailed breakdown of performance results on MMTU, at the dataset level. Reasoning models indeed outperform chat models in many cases, though chat models are better on knowledge-centric tasks like CTA, where we find reasoning models have a tendency to hallucinate (e.g., type names that do not exist in knowledge bases).

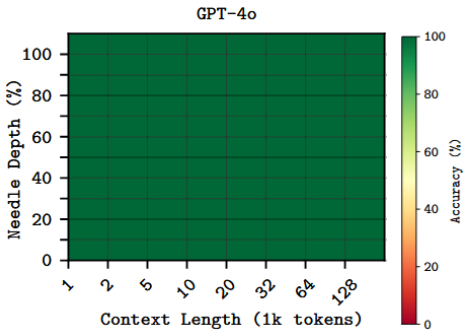

Figure 11: The needle-in-a-haystack test used in NLP context. Each cell in the heatmap corresponds to a test to retrieve a simple fact planted in a document, for a given document length and retrieval depth. Frontier models like GPT-4o and Gemini-2.5 can now typically achieve perfect accuracy, as shown by the all-green heatmap (results from [104]).

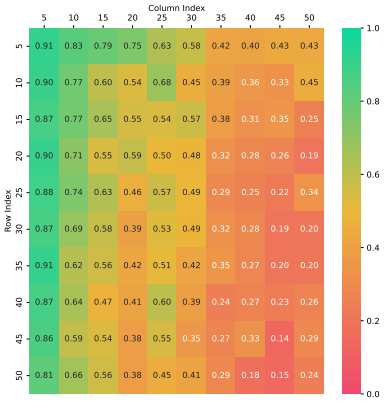

Figure 12: Our table-based needle-in-a-haystack test, where frontier models like GPT-4o continue to struggle to retrieve simple facts from large tables, especially with more columns (notice the asymmetry in the heatmap, where an increased column-index makes the task significantly harder).

C Long-context table understanding: A case study of "needle in a haystack in tables"

In this section, we take a closer look at one simple task in MMTU that we refer to as "needle in a haystack in tables", to more deeply analyze the problem of table understanding with long table context (large tables with many rows and columns).

Recall that "Needle-in-a-haystack (NIH)" [3, 104], is a popular test traditionally used to evaluate long-context LLM's ability to retrieve a simple fact (a needle) from a long document context (e.g., 100K or 1M tokens). Recent advances in frontier long-context LLMs have made this task almost irrelevant, as most frontier models such as GPT-4o, Llama-4, and Gemini-2.5 can now score perfectly on NIH [91, 111, 26, 14], like the heatmap in Figure 11 would show. Here the all-green heatmap indicates that this particular LLM under test, GPT-4o, can perfectly retrieve a needle planted at varying depth (y-axis), within documents of varying lengths (x-axis), with 100% accuracy.

We adapt NIH to the table setting, and study the table-version of the "needle in a haystack", which we call "needle-in-a-haystack-in-table" (NIHT), that tests LLM's ability to retrieve a simple fact (needle) within a cell of a table. Specifically, in NIHT, we randomly sample 100 real tables with at least 50 rows and columns, and ask LLMs to retrieve a simple fact randomly planted at row-i and column-j of each table, repeated over all (i, j) positions. LLM responses at different (i, j) positions can then be compared with the ground-truth (the cell value at those positions) to measure accuracy.

We would like to highlight that NIHT is the simplest possible task, for table understanding in long table context, as it performs a simple retrieval with no additional steps. Any real table tasks with long table context (e.g., Table QA on large tables) will necessarily be at least as hard as NIHT, as those tasks will need to first retrieve/identify relevant cells in long table context, before further processing (reasoning, calculation or coding) can be performed, making NIHT a good case study for long-context table understanding.

The results of NIHT are illustrated also using a heatmap in Figure 12. As we can see, compared to Figure 11, today's frontier LLMs still face substantial challenges in correctly identifying a cell value within a two-dimensional table, like indicated by the red cells in the heatmap. This is especially true when we increase the number of columns (e.g., with more than 25 columns, LLM accuracy quickly falls below 0.5). Furthermore, we observe a *strong asymmetry* in the heatmap – e.g., with 50 rows and 5 columns we still have a reasonable accuracy of 0.81 (at the lower-left corner of the heatmap), but with 5 rows and 50 columns the accuracy drops to a lowly 0.43 (at the top-right corner), underscoring

the challenge LLMs face when reading “vertically” in the column direction, which are consistent with our intuition that LLMs pre-trained on one-dimensional natural-language texts are less effective in reading two-dimensional tables “vertically”, like was also observed in [88, 110].